Time awareness

2026-08-16. Every number below is measured from `data_tables/real`, not quoted from config — that is the point of the change this documents.

The system used to carry ONE date: a `cut_off_date` constant in the pillar YAML, injected into every specialist as *"interpret ALL time-window language relative to this date"*. It is gone. Time now comes from the data, per case and per column, and three different questions resolve three different ways.

(Diagrams are plain ASCII on purpose: box-drawing glyphs are outside Courier's WinAnsi encoding and render substituted or blank in the PDF.)

Why one constant could not work

It was wrong in three independent directions at once.

```
ONE PILLAR CONSTANT                 WHAT THE DATA SAYS
cut_off_date: 2025-07-01

                                    case 11854808010   case 366132845011
     |                              ----------------   -----------------
     +-- every CASE ..............  ends 2025-07-01    ends 2025-06-30
     |                              (they differ)
     |
     +-- every TABLE .............  bureau  Jul'2025   bureau  Jun'2025
     |                              spends  07-01      spends  07-01
     |                              wcc     12-05      wcc     02-17
     |                              (five months apart INSIDE one case)
     |
     +-- forever .................  nothing in the data can contradict it,
                                    so it goes stale silently
```

The third is the one that bit. Nothing failed when it drifted; answers just started naming a month the column never reached, and read as confident.

Where "recent" comes from

Three sources, in strict order. The specialist gets all of it in the round-1 inventory as `§ DATA COVERAGE`.

```
a question mentions time
     |
     +-- 1. is a window ALREADY established?
     |      the reviewer named one, or a prior turn / another
     |      specialist pinned one (kb_lookup, episodic context)
     |         -> USE IT, and say so
     |      an anchor you derive that contradicts the window under
     |      discussion is the wrong answer to the right question
     |
     +-- 2. does the question name a METRIC or TABLE?
     |         -> use THAT column's own dates, never the case anchor
     |
     |      point  ("the latest FICO")     -> its last row
     |      window ("transactions last      -> its last period WITH DATA,
     |               month")                  plus what that period covers
     |
     +-- 3. otherwise it is GENERAL
            ("recent behaviour", "what stands out lately")
               -> the single CASE CUT-OFF, so every specialist on the
                  case measures the same period and the answers can
                  be compared. Say which window you used.
```

Step 2 before step 3 is load-bearing. Routing a table-scoped window through the shared anchor lands a month early whenever the two disagree: on case 366132845011 the anchor is 2025-06-30 while `spends` runs to 2025-07-01.

What a column REACHES is not what it says

`case_date_coverage()` returns, per date column: `first`, `last` (both verbatim, because that is what a filter has to be handed), plus `grain` and `covered_to`.

The last two exist because `_date_key` flattens `June'2025` to `(2025, 6, 1)`. Read literally that says the table reaches June 1st. It does not — a monthly row covers all of June.

```
column                      grain   last          covered_to
--------------------------  -----   -----------   ----------
bureau_data.month           month   June'2025     2025-06-30    <- rounded out
spends_data.Date            day     2025-07-01    2025-07-01
spends_data.Month           month   July'2025     2025-07-31    <- a LABEL
modelling_data_transaction day      2025-06-30    2025-06-30
  .trans_dt
```

Grain is decided by the data, not the name: a column whose every value lands on the 1st has no day resolution.

Three month-grain columns reading as "June 1st" had been dragging case 366132845011's anchor to 2025-06-01 — behind its own transaction table, which holds every day through 2025-06-30.

Inside a table, a dated column outranks a rollup label. `spends_data` carries `Date` (real days, to 2025-07-01) beside a `Month` label (July'2025). Rounding the label out to 2025-07-31 would claim three weeks the table does not hold, so where a table has day resolution, that column speaks for it.

CASE CUT-OFF: one anchor, derived

`case_cut_off()` takes each table's furthest-reaching column, then the MEDIAN across tables. Computed across the WHOLE case, never the specialist's own subset — derived per specialist it would differ per specialist, which is the divergence it exists to remove.

```
case 11854808010                    case 366132845011
----------------                    -----------------
payments_data       2025-06-28      strategy            2025-02-15  <- outlier
modelling_data      2025-06-30      wcc                 2025-02-17  <- outlier
score_drivers_data  2025-06-30      bureau_data         2025-06-30
spends_data         2025-07-01  <=  modelling_data      2025-06-30
bureau_data         2025-07-31      modelling_data_txn  2025-06-30  <=
crossbu_cards       2025-07-31      score_drivers_data  2025-06-30
wcc                 2025-12-05      payments_data       2025-07-01
demographics_data   2025-12-31      score_drivers_txn   2025-07-01
                                    spends_data         2025-07-01
ANCHOR  2025-07-01                  crossbu_cards       2025-07-31
                                    ANCHOR  2025-06-30
```

The median, not the min or the max, and both extremes show why: they are held by tables that are not time series. `strategy` has two rows and stops in February; `demographics` holds a single date; `wcc` is an event log that runs five months PAST every transactional table on one case and four months SHORT on the other. A min anchors the whole team to the sparsest event, a max to a window with no spend in it. The median lands where the case's continuous data actually ends, with no hand-maintained list of which tables count.

The period asked for vs the period compared against

The distinction that is easy to collapse, and answering the wrong one produces a number that looks better and means something else.

```
"how many transactions last month"     case 366132845011

  July  2025    22 transactions    <- THE ANSWER. The month asked about.
                                      Disclose that it holds 1 of 31 days.
  June  2025   478 transactions    <- the COMPARISON. What "up" or "down"
                                      is measured against. NOT a substitute.
  May   2025   682 transactions    <- a third month entirely; what routing
                                      through the case anchor used to give
```

Report the period the reviewer named, then say what it covers. Swapping in the last COMPLETE period answers a different question.

A short period is disclosed, never swapped

An export cut mid-period leaves a stub, and an additive measure over a stub is not small, it is INCOMPLETE. On both real cases `spends_data.Date` stops on 2025-07-01 — one day of thirty-one:

```
case 11854808010          case 366132845011
2025-05    99 txn         2025-05   682 txn
2025-06   120 txn         2025-06   478 txn
2025-07     9 txn  <--    2025-07    22 txn  <--   a 95% "collapse" that is
                                                   the export, not the customer
```

Two mechanisms, because a trend can see its neighbours and a count cannot:

	tool	what it does
trend	`summary.last_bucket_partial`	names the stub, hands over `summary.last_complete`, and measures slope / pct-change through the last WHOLE bucket
count / sum	`coverage_note`	says the table stops mid-period and that the figure is incomplete, not low

`summary.last` is still reported untouched — the flag describes the edge, it does not hide it.

Sparsity triggers it, day coverage only explains. Reaching the end of a period is not the same as covering it: a complete March whose last transaction fell on the 28th covers 28 of 31 days, and a table with one payment a month covers one. Both are whole periods. What a cut-off actually leaves is a record count far under the series norm — 9 against a median of 268, 22 against 366 — and that reads the same at day grain and at month grain.

The count-side note keys off the TABLE's day-grain reach rather than the filtered column, so a filter on a month LABEL is covered too: `Month eq July'2025` also returns 22, and the label cannot know July has one day.

Snapshots are exempt. `bureau_data` carries one row per month, so `max(FICO)` at the edge is a COMPLETE observation. Flagging it would suppress the right answer to "the most recent FICO score". Only sums and counts are diluted.

Dates sort by calendar

`sort_by=<date col>, sort_desc=True, limit=1` is the natural way to ask for the newest row, and it used to sort as TEXT. "September" is alphabetically last among the month names — ahead of every October, November and December, and ahead of any later year:

```
"what is the latest FICO score"    case 366132845011

before   September'2024   703      alphabetical: wrong month AND wrong year
after    June'2025        764      the last row of the column
```

Not specific to the apostrophe format the real bureau data uses. `MM/DD/YYYY` puts 03/2025 ahead of 12/2024; `strategy.Date` (`M/D/YY H:MM`) inverts outright.

`_row_sort_key` orders numbers, then dates, then text. Numbers are tested FIRST, which is what stops `_date_key`'s bare-integer branches (4-digit year, 5-digit Excel serial) from reading a FICO score of 764 as the year 764. `query_table` echoes `sort` as `"<col> desc (chronological)"` when the column sorted as dates — **if that marker is missing on a date sort, the column did not parse and the top row is alphabetical.**

Worked answers, case 366132845011

question	resolves via	answer
"the latest FICO"	point, bureau's own column	**764**, June'2025
"how many transactions last month"	window, `spends`' own dates	**22**, July 2025 (1 of 31 days; June was 478)
"recent behaviours"	general -> CASE CUT-OFF	anchored to **2025-06-30**

Note the first two disagree on the month and both are right. `bureau` stops at June'2025 while `spends` runs into July, so a point question about FICO and a window question about transactions legitimately land in different months. An answer that mixes them should say which month each figure is from.

Limits

- **The anchor is validated on two cases.** A case whose tables stop at genuinely scattered points may land the median somewhere neither obvious nor wrong. `case_cut_off()["per_table"]` shows the working.
- **Grain is inferred, not declared.** A day-grain column whose values all happen to fall on the 1st would read as month grain. Impossible at 500+ distinct dates, possible on a two-row table.
- **A format `_date_key` cannot parse falls to the text tier silently.** The `(chronological)` marker on `sort` is the only tell; nothing raises.
- `covered_to` **describes the export, not the world.** A table missing because an upstream feed failed looks identical to one that legitimately ends early.
- **Counting transactions defaults to** `spends`, which is the more current view — but it and the transaction-level model tables do NOT hold the same rows: the model side also carries auths and declines that never settled, so it has MORE rows over a SHORTER span (9,639 vs 8,888 on case 366132845011). A count that must include unsettled attempts has to name its table.